Федеральное государственное автономное образовательное учреждение высшего образования Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники

Распределенные системы хранения данных

Лабораторная работа №3

Вариант № 309658

Выполнили: студенты группы P3308,
Васильев Н. А.

Елисеев К. И.

Преподаватель: Заболотняя О. М.

Санкт-Петербург 2026

Цель работы

Ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД (например, через psql), использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере **не менее, чем двух** таблиц, столбцов, строк, транзакций и клиентских сессий.

Текст задания

Этап 1. Конфигурация

Развернуть postgres на двух узлах в режиме трансляции логов. Не использовать дополнительные пакеты. Продемонстрировать доступ в режиме чтение/запись на основном сервере. Продемонстрировать, что новые данные синхронизируются на резервный сервер.

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:

Симулировать переполнение дискового пространства на основном узле - заполнить всё свободное пространство раздела с PGDATA “мусорными” файлами.

2.3 Обработка:

- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Восстановление

- Восстановить работу основного узла - откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

Выполнение

На основном узле изменим postgresql.conf, добавив следующие параметры:

```
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = '128MB'
hot_standby = on
archive_mode = on
archive_command = 'test ! -f /var/lib/postgresql/archive/%f && cp %p
/var/lib/postgresql/archive/%f'
full_page_writes = on
wal_log_hints = on
logging_collector = on
log_directory = '/var/lib/postgresql/pglogs'
log_filename = 'postgresql.log'
```

Репликационный пользователь создаётся на основном узле:

```
CREATE ROLE repl WITH REPLICATION LOGIN PASSWORD 'replpass';
```

Проверим параметров и состояния основного узла:

```
SHOW wal_level;
SHOW max_wal_senders;
SHOW hot_standby;
SHOW wal_log_hints;
SHOW full_page_writes;
SELECT pg_is_in_recovery();
```

```
 wal_level
-----------
 replica
(1 row)

 max_wal_senders
-----------------
 10
(1 row)

 hot_standby
-------------
 on
(1 row)

 wal_log_hints
---------------
 on
(1 row)

 full_page_writes
------------------
 on
(1 row)

 pg_is_in_recovery
-------------------
 f
(1 row)
```

Резервный узел создаётся не как отдельная новая база, а как физическая копия основного узла. Для этого используется pg_basebackup

```
pg_basebackup \
  -h pg-primary \
  -p 5432 \
  -U repl \
  -D "$PGDATA" \
  -R \
  -c fast \
  -X stream \
  -C \
  -S standby_slot
```

В данном случае backup берётся с контейнера pg-primary. Подключение выполняется от имени пользователя repl. Это специальный пользователь для репликации. С помощью флага -R автоматически настраивает полученный кластер как standby. После backup создаётся файл standby.signal, а в postgresql.auto.conf добавляются параметры подключения к primary.

Указав -X stream мы передаём WAL-журналы потоком во время backup. Это нужно, чтобы полученная копия была консистентной и могла стартовать.

Создаем replication slot через -C на основном сервере. Задаём имя replication slot: standby_slot. Этот slot нужен, чтобы primary не удалял WAL-файлы, которые ещё нужны standby.

Проверка состояния резервного узла:

```
SELECT pg_is_in_recovery();
SELECT status, receive_start_lsn, written_lsn, flushed_lsn, latest_end_lsn,
slot_name, sender_host, sender_port
FROM pg_stat_wal_receiver;
```

```
 pg_is_in_recovery
-------------------
 t
(1 row)

  status   | receive_start_lsn | written_lsn | flushed_lsn | latest_end_lsn |
slot_name   | sender_host | sender_port
-----------+-------------------+-------------+-------------+----------------
+--------------+-------------+-------------
 streaming | 0/3000000         | 0/30437F8   | 0/30437F8   | 0/30437F8      |
standby_slot | pg-primary  |        5432
(1 row)
```

Резервный узел успешно создан из физической копии основного сервера.

pg_is_in_recovery() = true подтверждает, что узел работает как standby.

pg_stat_wal_receiver.status = streaming подтверждает, что standby получает WAL с pg-primary в режиме потоковой репликации.

Создадим таблицы:

```
CREATE TABLE IF NOT EXISTS accounts (
  id          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  owner_name  text NOT NULL,
  balance     numeric(12, 2) NOT NULL CHECK (balance >= 0)
);

CREATE TABLE IF NOT EXISTS transfers (
  id               bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  from_account_id  integer NOT NULL REFERENCES accounts(id),
  to_account_id    integer NOT NULL REFERENCES accounts(id),
  amount           numeric(12, 2) NOT NULL CHECK (amount > 0),
```

```
  note              text NOT NULL DEFAULT '',
  created_at        timestamptz NOT NULL DEFAULT now()
);
```

Начальное наполнение:

```
INSERT INTO accounts (owner_name, balance) VALUES
  ('Alice', 900.00),
  ('Bob', 900.00),
  ('Charlie', 1250.00);

INSERT INTO transfers (from_account_id, to_account_id, amount, note) VALUES
  (1, 2, 100.00, 'initial transfer'),
  (2, 3, 50.00, 'seed transfer');
```

Проверим на pg-primary:

```
SELECT * FROM accounts ORDER BY id;
SELECT * FROM transfers ORDER BY id;
```

```
 id | owner_name | balance
----+------------+---------
  1 | Alice      |  900.00
  2 | Bob        |  900.00
  3 | Charlie    | 1250.00
(3 rows)

 id | from_account_id | to_account_id | amount |       note       |
created_at
----+-----------------+---------------+--------+------------------+----------
--------------------
  1 |               1 |             2 | 100.00 | initial transfer | 2026-04-
23 20:21:26.226727+00
  2 |               2 |             3 |  50.00 | seed transfer    | 2026-04-
23 20:21:26.226727+00
(2 rows)
```

Проверим на pg-standby:

```
SELECT * FROM accounts ORDER BY id;
SELECT * FROM transfers ORDER BY id;
```

```
 id | owner_name | balance
----+------------+---------
  1 | Alice      |  900.00
  2 | Bob        |  900.00
  3 | Charlie    | 1250.00
(3 rows)

 id | from_account_id | to_account_id | amount |       note       |
created_at
----+-----------------+---------------+--------+------------------+----------
--------------------
  1 |               1 |             2 | 100.00 | initial transfer | 2026-04-
23 20:21:26.226727+00
  2 |               2 |             3 |  50.00 | seed transfer    | 2026-04-
23 20:21:26.226727+00
(2 rows)
```

Проверим read/write на основном узле и синхронизации на резервный

Проверяем, что основной узел pg-primary принимает операции записи, а резервный узел pg-standby получает эти изменения через потоковую репликацию.

На основном узле выполняется транзакция перевода:

```
docker compose exec -T "$CLIENT_SERVICE" \
  env PGPASSWORD="$LAB_PASS" PGAPPNAME="transfer-script" \
  psql \
    -v ON_ERROR_STOP=1 \
    -v from_id="1" \
    -v to_id="2" \
    -v amount="150.00" \
    -v note="report-primary-transfer" \
    -h "pg-primary" \
    -U "$LAB_USER" \
    -d "$LAB_DB" \
    -f /workspace/sql/transfer.sql
```

Файл `transer.sql`:

```
\if :{?from_id}
\else
\set from_id 1
\endif

\if :{?to_id}
\else
\set to_id 2
\endif

\if :{?amount}
\else
\set amount 150.00
\endif

\if :{?note}
\else
\set note demo-transfer
\endif

BEGIN;
UPDATE accounts
SET balance = balance - :amount
WHERE id = :from_id;
```

```
UPDATE accounts
SET balance = balance + :amount
WHERE id = :to_id;

INSERT INTO transfers (from_account_id, to_account_id, amount, note)
VALUES (:from_id, :to_id, :amount, :'note');
COMMIT;

SELECT * FROM accounts ORDER BY id;
SELECT * FROM transfers ORDER BY id;
```

```
[2026-04-23 23:22:10] Running transfer on pg-primary.
BEGIN
UPDATE 1
UPDATE 1
INSERT 0 1
COMMIT
```

Получается:

- транзакция началась;
- обновлена одна строка отправителя;
- обновлена одна строка получателя;
- добавлена одна запись в таблицу переводов;
- транзакция успешно зафиксирована.

```
 id | owner_name | balance
----+------------+---------
  1 | Alice      |  750.00
  2 | Bob        | 1050.00
  3 | Charlie    | 1250.00
(3 rows)

 id | from_account_id | to_account_id | amount |          note           |
created_at
----+-----------------+---------------+--------+-------------------------+---
----------------------------
  1 |               1 |             2 | 100.00 | initial transfer        |
2026-04-23 20:21:26.226727+00
  2 |               2 |             3 |  50.00 | seed transfer           |
2026-04-23 20:21:26.226727+00
  3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
(3 rows)
```

Вывод показывает, что операция записи на pg-primary успешно выполнена.

Дополнительно фиксируем ещё одну запись и проверяем, что она доехала до pg-standby:

```
docker compose exec -T "$CLIENT_SERVICE" \
  env PGPASSWORD="$LAB_PASS" PGAPPNAME="transfer-script" \
  psql \
```

```
    -v ON_ERROR_STOP=1 \
    -v from_id="1" \
    -v to_id="3" \
    -v amount="25.00" \
    -v note=" report-sync-proof" \
    -h "pg-primary" \
    -U "$LAB_USER" \
    -d "$LAB_DB" \
    -f /workspace/sql/transfer.sql
```

Эта команда выполняет перевод: `Alice -> Charlie, 25.00`

Цель этой операции – создать отдельную запись, по которой затем удобно проверить, что изменение дошло до `pg-standby`

```
[2026-04-23 23:23:49] Running transfer on pg-primary.
BEGIN
UPDATE 1
UPDATE 1
INSERT 0 1
COMMIT
```

Операция успешно выполнена.

```
 id | owner_name | balance
----+------------+---------
  1 | Alice      |  725.00
  2 | Bob        |  975.00
  3 | Charlie    | 1350.00
(3 rows)

 id  | from_account_id | to_account_id | amount |          note           |
created_at
-----+-----------------+---------------+--------+-------------------------+--
-----------------------------
   1 |               1 |             2 | 100.00 | initial transfer        |
2026-04-23 20:21:26.226727+00
   2 |               2 |             3 |  50.00 | seed transfer           |
2026-04-23 20:21:26.226727+00
   3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
 133 |               2 |             3 |  75.00 | report-after-failover   |
2026-04-23 20:22:37.963146+00
 134 |               1 |             3 |  25.00 | report-sync-proof       |
2026-04-23 20:23:49.489785+00
(5 rows)
```

Это подтверждает, что на основном узле была выполнена новая операция записи.

Проверим на pg-standby:

```
SELECT * FROM accounts ORDER BY id;
SELECT * FROM transfers ORDER BY id DESC LIMIT 3;
```

```
 id | owner_name | balance
----+------------+---------
```

```
  1 | Alice      |  725.00
  2 | Bob        |  975.00
  3 | Charlie    | 1350.00
(3 rows)

 id  | from_account_id | to_account_id | amount |          note           |
created_at
-----+-----------------+---------------+--------+-------------------------+--
----------------------------
 134 |               1 |             3 |  25.00 | report-sync-proof       |
2026-04-23 20:23:49.489785+00
 133 |               2 |             3 |  75.00 | report-after-failover   |
2026-04-23 20:22:37.963146+00
   3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
(3 rows)
```

Если на pg-standby видна запись: 134 | 1 | 3 | 25.00 | report-sync-proof

значит изменение, выполненное на pg-primary, успешно доставлено на резервный узел.

Клиентские подключения

Для демонстрации одновременной работы нескольких клиентов были открыты три отдельные клиентские сессии:

```
make client1
make client2
make client3
```

Каждая команда открывает отдельное подключение к PostgreSQL от имени пользователя labuser. Для различения сессий используется разное значение application_name.

В каждой клиентской сессии была запущена длительная команда:

```
SELECT pg_sleep(20);
```

Она удерживает подключение активным 20 секунд, чтобы в это время можно было зафиксировать состояние клиентских сессий через pg_stat_activity.

Снимок pg_stat_activity с тремя одновременными клиентскими сессиями:

```
SELECT pid, usename, application_name, client_addr, backend_type, state,
wait_event_type, wait_event, xact_start, query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY application_name, pid;
```

```
Pager usage is off.
 pid | usename | application_name |  client_addr  |  backend_type  | state  |
wait_event_type | wait_event |          xact_start           |
query
-----+---------+------------------+---------------+----------------+--------
+-----------------+------------+-------------------------------+-------------
-----------------------
 281 | labuser | client-1         | 192.168.117.4 | client backend | active |
Timeout         | PgSleep    | 2026-04-23 20:22:22.666604+00 | SELECT
pg_sleep(20);
 283 | labuser | client-2         | 192.168.117.4 | client backend | active |
Timeout         | PgSleep    | 2026-04-23 20:22:22.668186+00 | SELECT
pg_sleep(20);
 282 | labuser | client-3         | 192.168.117.4 | client backend | active |
Timeout         | PgSleep    | 2026-04-23 20:22:22.666633+00 | SELECT
pg_sleep(20);
```

```
 285 | labuser | psql             | 192.168.117.4 | client backend | active |
|            | 2026-04-23 20:22:28.664488+00 | SELECT pid,
+
     |         |                  |               |                |        |
|            |                               |        usename,
+
     |         |                  |               |                |        |
|            |                               |        application_name,
+
     |         |                  |               |                |        |
|            |                               |        client_addr,
+
     |         |                  |               |                |        |
|            |                               |        backend_type,
+
     |         |                  |               |                |        |
|            |                               |        state,
+
     |         |                  |               |                |        |
|            |                               |        wait_event_type,
+
     |         |                  |               |                |        |
|            |                               |        wait_event,
+
     |         |                  |               |                |        |
|            |                               |        xact_start,
+
     |         |                  |               |                |        |
|            |                               |        query
+
     |         |                  |               |                |        |
|            |                               | FROM pg_stat_activity
+
     |         |                  |               |                |        |
|            |                               | WHERE datname =
current_database()+
     |         |                  |               |                |        |
|            |                               | ORDER BY application_name,
pid;
(4 rows)
```

Из результата видно, что одновременно открыты три клиентские сессии, все они подключены пользователем labuser и находятся в состоянии active.

Симуляция переполнения диска и failover

На этом этапе искусственно заполняется раздел, на котором расположен каталог данных PostgreSQL (PGDATA). Цель – вызвать отказ основного узла из-за невозможности записывать WAL и файлы данных.

```
df -h "$PGDATA"

mkdir -p "$PGDATA/fill"

dd if=/dev/zero of="$PGDATA/fill/ballast.bin" bs=1M status=progress
conv=fsync

df -h "$PGDATA"
```

Записывает нули в файл `ballast.bin` до тех пор, пока на разделе не закончится свободное место.

```
[2026-04-23 23:22:36] Primary filesystem usage before fill:
Filesystem      Size  Used Avail Use% Mounted on
```

```
tmpfs           256M   78M  179M  31% /var/lib/postgresql/data/pgdata
[2026-04-23 23:22:36] Writing ballast file into PGDATA until ENOSPC.
dd: error writing '/var/lib/postgresql/data/pgdata/fill/ballast.bin': No
space left on device
179+0 records in
178+0 records out
186699776 bytes (187 MB, 178 MiB) copied, 0.0346244 s, 5.4 GB/s
dd exit code: 1
[2026-04-23 23:22:36] Primary filesystem usage after fill:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           256M  256M     0 100% /var/lib/postgresql/data/pgdata
```

Ошибка `No space left on device` означает, что свободное место на разделе закончилось. `dd exit code: 1` показывает, что команда завершилась с ошибкой, что в данном случае ожидаемо: файл записывался до полного заполнения диска.

После заполнения раздела с PGDATA была выполнена попытка записать новые данные в БД:

```
INSERT INTO transfers (from_account_id, to_account_id, amount, note)
SELECT 1, 2, 1.00, 'enospc-check'
FROM generate_series(1, 1000);
```

```
[2026-04-23 23:22:37] Triggering a write on the full primary to force
PostgreSQL errors.
ERROR:  could not extend file "base/16386/16417_fsm": No space left on device
HINT:  Check free disk space.
[2026-04-23 23:22:37] Write failed as expected. Relevant primary log lines:
2026-04-23 20:22:37.062 UTC [312] labuser@labdb ERROR:  could not extend file
"base/16386/16417_fsm": No space left on device
```

Получаем ошибку: `No space left on device`, которая означает, что PostgreSQL не смог расширить один из файлов данных, потому что на файловой системе закончилось свободное место.

После переполнения диска основной узел pg-primary потерял возможность выполнять операции записи. Поэтому выполняется переключение на резервный узел pg-standby.

```
docker compose exec -T pg-primary gosu postgres pg_ctl -D "$PGDATA" stop -m
immediate || true
```

Переводим резервный узел из режима recovery в режим primary.

После этого `pg-standby` перестаёт быть резервным узлом и становится новым основным сервером.

```
docker compose exec -T pg-standby gosu postgres pg_ctl -D "$PGDATA" promote

docker compose exec -T pg-standby env PGPASSWORD="$LAB_PASS" \
psql -U "$LAB_USER" -d "$LAB_DB" \
-c "SELECT pg_is_in_recovery();"

docker compose exec -T client env PGPASSWORD="$LAB_PASS" \
psql -h pg-standby -U "$LAB_USER" -d "$LAB_DB" \
```

```
-c "SELECT now() AS failover_completed_at;"
```

```
 [2026-04-23 23:22:37] Primary error log excerpt before failover.
2026-04-23 20:22:37.062 UTC [312] labuser@labdb ERROR:  could not extend file
"base/16386/16417_fsm": No space left on device
[2026-04-23 23:22:37] Stopping failed primary if it is still running.
waiting for server to shut down.... done
server stopped
[2026-04-23 23:22:37] Promoting standby.
waiting for server to promote.... done
server promoted
[2026-04-23 23:22:37] Standby is now primary.
 pg_is_in_recovery
-------------------
 f
(1 row)

     failover_completed_at
-------------------------------
 2026-04-23 20:22:37.860122+00
(1 row)
```

Этот запрос показывает, что клиент уже может подключиться к новому primary pg-standby.

После переключения запись выполняется уже на новом основном узле. Она выполняет транзакцию перевода.

```
docker compose exec -T client \
  env PGPASSWORD="$LAB_PASS" PGAPPNAME="transfer-after-failover" \
  psql \
    -v ON_ERROR_STOP=1 \
    -v from_id="2" \
    -v to_id="3" \
    -v amount="75.00" \
    -v note="report-after-failover" \
    -h "pg-standby" \
    -U "$LAB_USER" \
    -d "$LAB_DB" \
    -f /workspace/sql/transfer.sql
```

```
[2026-04-23 23:22:37] Running transfer on pg-standby.
BEGIN
UPDATE 1
UPDATE 1
INSERT 0 1
COMMIT
```

Это означает:

- транзакция началась;
- баланс отправителя обновлён;
- баланс получателя обновлён;

- запись о переводе добавлена;
- транзакция успешно зафиксирована.

```
 id | owner_name | balance
----+------------+---------
  1 | Alice      |  750.00
  2 | Bob        |  975.00
  3 | Charlie    | 1325.00
(3 rows)

 id  | from_account_id | to_account_id | amount |          note           |
created_at
-----+-----------------+---------------+--------+-------------------------+--
----------------------------
   1 |               1 |             2 | 100.00 | initial transfer        |
2026-04-23 20:21:26.226727+00
   2 |               2 |             3 |  50.00 | seed transfer           |
2026-04-23 20:21:26.226727+00
   3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
 133 |               2 |             3 |  75.00 | report-after-failover   |
2026-04-23 20:22:37.963146+00
(4 rows)
```

В таблице переводов появилась новая запись: `133 | 2 | 3 | 75.00 | report-after-failover`

Это подтверждает, что после failover новый основной узел `pg-standby` принимает операции записи.

Восстановление бывшего primary и возврат исходной топологии

После failover новым основным узлом стал pg-standby. Бывший основной узел pg-primary нужно восстановить и подключить обратно уже как резервный узел.

```
SELECT pg_is_in_recovery();
```

```
pg_ctl -D "$PGDATA" stop -m immediate || true
```

Проверяем, существует ли replication slot с именем `primary_slot`. Если slot уже есть, он удаляется. Это нужно, чтобы при новом pg_basebackup можно было создать slot заново и избежать конфликта имён.

```
DO $$
BEGIN
  IF EXISTS (
    SELECT 1
    FROM pg_replication_slots
    WHERE slot_name = 'primary_slot'
  ) THEN
```

```
    PERFORM pg_drop_replication_slot('primary_slot');
  END IF;
END $$;

mkdir -p "$PGDATA" /var/lib/postgresql/pglogs /var/lib/postgresql/archive
find "$PGDATA" -mindepth 1 -maxdepth 1 -exec rm -rf {} + || true
rm -f /var/lib/postgresql/pglogs/postgresql.log
chown -R postgres:postgres \
    "$PGDATA" \
    /var/lib/postgresql/pglogs \
    /var/lib/postgresql/archive
```

Создаем новую копию с текущего primary.

```
pg_basebackup \
    -h pg-standby \
    -p 5432 \
    -U repl \
    -D "$PGDATA" \
    -R \
    -c fast \
    -X stream \
    -C \
    -S primary_slot
```

В файл `postgres.auto.conf` добавляются параметры:

```
primary_conninfo = '\''host=pg-standby port=5432 user=repl password=replpass application_name=pg-primary'\''
primary_slot_name = '\''primary_slot'\''
```

primary_conninfo указывает, как бывший primary теперь должен подключаться к новому primary pg-standby.

Запускаем бывший primary как standby:

```
pg_ctl \
    -D "$PGDATA" \
    -l /var/lib/postgresql/pglogs/postgresql.log \
    start
```

Так как в каталоге данных есть `standby.signal` и `primary_conninfo`, сервер запускается не как primary, а как standby.

```
SELECT pg_is_in_recovery();
```

Проверим получения WAL

```
SELECT
  status,
  receive_start_lsn,
  written_lsn,
  flushed_lsn,
  latest_end_lsn,
  slot_name
FROM pg_stat_wal_receiver;
```

```
[2026-04-23 23:22:38] Cleaning failed primary.
DO
waiting for server to start.... done
server started
[2026-04-23 23:22:38] Former primary is back as standby.
 pg_is_in_recovery
-------------------
 t
(1 row)
```

t означает true: сервер работает в режиме recovery, то есть теперь это standby.

```
  status   | receive_start_lsn | written_lsn | flushed_lsn | latest_end_lsn |
slot_name
-----------+-------------------+-------------+-------------+----------------
+--------------
 streaming | 0/5000000         |             | 0/5000000   |                |
primary_slot
(1 row)
```

Бывший primary получает WAL от нового primary. Для репликации используется replication slot `primary_slot`.

Возвращаем исходную топологию:

После восстановления бывшего primary узлы временно находятся в такой схеме:

```
pg-standby = текущий primary
pg-primary = standby
```

На этом этапе выполняется обратное переключение, чтобы вернуть исходную топологию:

```
pg-primary = primary
pg-standby = standby
```

На `pg-primary`, который сейчас является standby, получаем последнюю применённую WAL-позицию:

```
SELECT pg_last_wal_replay_lsn();
```

Затем считаем отставание:

```
SELECT pg_wal_lsn_diff('<current_lsn>', '<replay_lsn>');
```

Если результат равен: 0 значит `pg-primary` полностью догнал текущий primary и его можно безопасно повышать обратно до primary.

```
pg_ctl -D "$PGDATA" stop
pg_ctl promote
```

выводит pg-primary из режима standby/recovery и делает его основным узлом.

После этого проверяется:

```
SELECT pg_is_in_recovery();
```

После promotion `pg-primary` снова стал основным узлом. Теперь `pg-standby` нужно заново подключить как standby.

Сначала на `pg-primary` удаляется старый replication slot, если он существует:

```
DO $$
BEGIN
  IF EXISTS (
    SELECT 1
    FROM pg_replication_slots
    WHERE slot_name = 'standby_slot'
  ) THEN
    PERFORM pg_drop_replication_slot('standby_slot');
  END IF;
END $$;
```

Если slot standby_slot уже существует, повторное создание backup с флагами -C -S standby_slot завершится ошибкой. Поэтому старый slot удаляется перед пересозданием standby.

Создание нового standby через pg_basebackup

```
pg_basebackup \
    -h pg-primary \
    -p 5432 \
    -U repl \
    -D "$PGDATA" \
    -R \
    -c fast \
    -X stream \
```

```
    -C \
    -S standby_slot
```

В `postgresql.auto.conf` добавляются строки:

```
primary_conninfo = 'host=pg-primary port=5432 user=repl password=replpass
application_name=pg-standby'
primary_slot_name = 'standby_slot'
```

Запускаем `pg-standby`:

```
pg_ctl \
-D "$PGDATA" \
-l /var/lib/postgresql/pglogs/postgresql.log \
start
```

```
[2026-04-23 23:22:38] Waiting for pg-primary to catch up with the current
primary.
[2026-04-23 23:22:39] Stopping current primary cleanly.
waiting for server to shut down.... done
server stopped
[2026-04-23 23:22:39] Promoting pg-primary back to primary.
waiting for server to promote.... done
server promoted
[2026-04-23 23:22:39] Re-seeding pg-standby from pg-primary.
DO
waiting for server to start.... done
server started
[2026-04-23 23:22:40] Original topology restored.
```

Это означает:

1. `pg-primary` догнал текущий primary.
2. Текущий primary `pg-standby` был корректно остановлен.
3. `pg-primary` был повышен обратно до роли primary.
4. `pg-standby` был пересоздан из нового primary.
5. Исходная топология восстановлена.

После возврата исходной топологии выполняется финальная проверка:

```
show_node() {
  local label=$1
  local service=$2
```

Проверка роли и WAL-позиции

```
SELECT inet_server_addr() AS server_addr,
       inet_server_port() AS server_port,
       pg_is_in_recovery() AS in_recovery,
       CASE
         WHEN pg_is_in_recovery() THEN pg_last_wal_replay_lsn()
         ELSE pg_current_wal_lsn()
       END AS visible_lsn;
```

В конце для каждого узла выполняется SQL-файл:

```
client_file "$service" "/workspace/sql/checks.sql"
```

Он проверяет состояние базы и таблиц:

```
SELECT * FROM accounts ORDER BY id;
SELECT * FROM transfers ORDER BY id;
```

Для pg-primary результат:

```
=== PRIMARY SERVICE: pg-primary ===
 server_addr | server_port | in_recovery | visible_lsn
-------------+-------------+-------------+-------------
             |             | f           | 0/70014D0
(1 row)
```

Это означает pg-primary = primary

К primary подключён standby-узел. Репликация активна и работает в асинхронном режиме. Standby получил, записал, сбросил на диск и применил WAL до той же позиции, что и primary. Отставания нет.

```
 application_name |   state   | sync_state | sent_lsn  | write_lsn |
flush_lsn | replay_lsn
------------------+-----------+------------+-----------+-----------+---------
--+------------
 pg-standby       | streaming | async      | 0/70014D0 | 0/70014D0 |
0/70014D0 | 0/70014D0
(1 row)

Pager usage is off.
 db_name | db_user |  server_addr  | server_port | in_recovery | visible_lsn
---------+---------+---------------+-------------+-------------+-------------
 labdb   | labuser | 192.168.117.2 |        5432 | f           | 0/70014D0
(1 row)

 id | owner_name | balance
----+------------+---------
  1 | Alice      |  750.00
  2 | Bob        |  975.00
  3 | Charlie    | 1325.00
(3 rows)

 id  | from_account_id | to_account_id | amount |          note           |
created_at
-----+-----------------+---------------+--------+-------------------------+--
----------------------------
   1 |               1 |             2 | 100.00 | initial transfer        |
2026-04-23 20:21:26.226727+00
   2 |               2 |             3 |  50.00 | seed transfer           |
2026-04-23 20:21:26.226727+00
   3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
 133 |               2 |             3 |  75.00 | report-after-failover   |
2026-04-23 20:22:37.963146+00
(4 rows)


=== STANDBY SERVICE: pg-standby ===
 server_addr | server_port | in_recovery | visible_lsn
-------------+-------------+-------------+-------------
             |             | t           | 0/70014D0
(1 row)
```

```
  status   | receive_start_lsn | written_lsn | flushed_lsn | latest_end_lsn |
slot_name
-----------+-------------------+-------------+-------------+----------------
+--------------
 streaming | 0/7000000         | 0/70014D0   | 0/70014D0   | 0/70014D0      |
standby_slot
(1 row)
```

Standby получает WAL от primary. Используется replication slot standby_slot.

```
Pager usage is off.
 db_name | db_user |  server_addr  | server_port | in_recovery | visible_lsn
---------+---------+---------------+-------------+-------------+-------------
 labdb   | labuser | 192.168.117.3 |        5432 | t           | 0/70014D0
(1 row)

 id | owner_name | balance
----+------------+---------
  1 | Alice      |  750.00
  2 | Bob        |  975.00
  3 | Charlie    | 1325.00
(3 rows)

 id  | from_account_id | to_account_id | amount |          note           |
created_at
-----+-----------------+---------------+--------+-------------------------+--
-----------------------------
   1 |               1 |             2 | 100.00 | initial transfer        |
2026-04-23 20:21:26.226727+00
   2 |               2 |             3 |  50.00 | seed transfer           |
2026-04-23 20:21:26.226727+00
   3 |               1 |             2 | 150.00 | report-primary-transfer |
2026-04-23 20:22:10.966735+00
 133 |               2 |             3 |  75.00 | report-after-failover   |
2026-04-23 20:22:37.963146+00
(4 rows)
```

Финальная проверка показала, что исходная топология восстановлена:

```
pg-primary = primary
pg-standby = standby
```

На `pg-primary` значение `pg_is_in_recovery()` равно `false`, а на `pg-standby` — `true`.

WAL-позиции на обоих узлах совпадают:

```
0/70014D0
```

Данные в таблицах `accounts` и `transfers` одинаковые на primary и standby. Это подтверждает, что после сбоя, failover, восстановления бывшего primary и возврата исходной топологии кластер работает исправно.

Вывод

В ходе работы был развёрнут отказоустойчивый стенд PostgreSQL из двух узлов: pg-primary и pg-standby. Была настроена потоковая репликация WAL, проверена запись на основном узле и синхронизация данных на резервный.

Далее был смоделирован отказ основного узла через переполнение раздела с PGDATA.

После ошибки No space left on device был выполнен failover: резервный узел был повышен до primary и успешно продолжил работу в режиме чтение/запись.

После этого бывший основной узел был восстановлен из pg_basebackup и подключён обратно как standby. Затем была выполнена обратная смена ролей, и исходная топология была восстановлена: pg-primary снова стал основным узлом, а pg-standby — резервным.

Финальная проверка показала, что репликация работает в режиме streaming, WAL-позиции совпадают, а данные на обоих узлах синхронизированы. Работа подтвердила практическое понимание настройки репликации, обработки отказа, failover и восстановления PostgreSQL-кластера.